

Builder Journal

Weekly Engineering & Performance Report

BIGGEST CHALLENGE

The hardest part of this week wasn't the computer vision itself — YOLO detection and ByteTrack/BoT-SORT tracking worked close to out-of-the-box. The hardest part was everything AROUND the model: getting a reliable pipeline running across three different frontends (CLI, Streamlit, a native Qt desktop app) on Windows, where the same code behaved differently depending on the OpenCV build, the webcam driver, and whether PyTorch could actually see the GPU.

Several full days went into environment and platform issues that had nothing to do with tracking logic — that ratio (mostly plumbing, not algorithms) was the real surprise of the week.

TRACKING ISSUES

- **Line-Crossing Accuracy:** Line-crossing counts were initially inaccurate — the default line-zone setting counted a crossing as soon as any corner of a bounding box touched the line, not when the object actually crossed. Fixed by switching to a center-point anchor, matching how the trajectory/direction logic already tracked objects.
- **Direction Detection Bug:** Direction detection had a subtle bug introduced and then caught by testing: the first attempt at 'smoothing' the direction calculation was mathematically identical to the un-smoothed version (a telescoping-sum coincidence), so it fixed nothing. Only building an actual test with injected jitter revealed this — the fix (comparing a window of points instead of two single points) genuinely reduced error.
- **Synchronization Crash:** A crash appeared specifically when a new object entered frame but the tracker hadn't yet confirmed an ID for it — the detection count and the label count went out of sync for that one frame. Fixed by making all detection fields empty together instead of patching just one field.
- **Model-Size Performance Paradox:** Assignment 5's performance testing surfaced a genuinely confusing result: the larger, theoretically more accurate model showed WORSE tracking-stability numbers than the smaller one. Still not fully resolved — most likely the bigger model is detecting extra small/hard objects the smaller one simply misses, which looks like instability in a proxy metric but might actually be more thorough detection. This needs labeled ground truth to settle properly.

PERFORMANCE BOTTLENECKS

- **CPU Limitation:** Running on CPU only, before GPU was properly configured: 5-10 FPS, clearly not real-time.

- **Silent PyTorch Default:** A silent trap: pip installs a CPU-only build of PyTorch on Windows by default, even with a real GPU present — this alone was the difference between 5 FPS and 25+ FPS, and it doesn't announce itself as a problem anywhere.
- **Webcam Buffer Backlog:** The webcam feed would build up a growing backlog of buffered frames whenever processing fell even briefly behind capture speed, showing up as a 'freeze, then jump ahead' stutter rather than smooth lag.
- **Streamlit Rendering Overhead:** Streamlit's live view was slower than it needed to be for two compounding reasons: sending full-resolution PNG frames over the browser connection, and forcing a full-page reload on every single frame update instead of refreshing just the video element.
- **Per-Frame Bookkeeping:** Assignment 5 found that once the model itself is fast, a roughly fixed 15-23ms of per-frame overhead (frame reading, Python-level bookkeeping) becomes the actual bottleneck — for the fastest model tested, over 60% of each frame's total time was this overhead, not the model.

OPTIMIZATION TECHNIQUES APPLIED

- **CUDA Reinstallation:** Reinstalled PyTorch with the correct CUDA build instead of the default CPU-only one — the single biggest speed win of the whole week.
- **Dynamic Runtime Adjustments:** Made inference resolution (imgsz) and half-precision (fp16) runtime-adjustable instead of fixed, so speed vs. accuracy can be tuned per situation instead of guessed at once.
- **Threaded Frame Acquisition:** Read the webcam in a background thread that always keeps only the latest frame, decoupling camera speed from processing speed and eliminating the buffer-backlog stutter.
- **Streamlit Stream Optimization:** Switched Streamlit's live frames to JPEG instead of PNG and moved the refresh into a partial-page update instead of a full reload.
- **Robust Failovers & Warmups:** Added a DirectShow-backend + short warmup-read sequence for opening the webcam, and a codec-fallback chain (plus FPS sanitization) for saving video — both came directly from real Windows-specific failures, not anticipated in advance.

LESSONS LEARNED

- **Infrastructure Costs Time:** On Windows specifically, environment and driver issues (camera backend, video codec availability, default CPU-only PyTorch) cost more real time this week than any tracking algorithm decision did.
- **Empirical Testing over Theory:** A fix that looks correct on paper can be mathematically equivalent to doing nothing — the direction-smoothing bug only surfaced because it was tested numerically, not just reasoned about.
- **Misleading Benchmarks:** Naive timing measurements can be misleading: a single unmeasured warmup frame, or GPU calls that return before finishing their work, can distort an average enough to draw the wrong conclusion if not explicitly checked for.

- **Unified Engine Architecture:** Keeping one shared engine behind three different frontends meant every bug fix and accuracy improvement applied everywhere at once, instead of needing to be re-applied (and re-debugged) three separate times.
- **Proxy Metric Caveats:** Proxy metrics (like ID-switch rate, used when there's no labeled ground truth) are genuinely useful but can point in a misleading direction — the counter-intuitive model-size result is a reminder to interpret them carefully rather than at face value.

WEEK 4 GOALS

- **Ground Truth Validation:** Resolve the open tracking-accuracy question from Assignment 5 with labeled ground truth or manual video review, instead of relying on proxies alone.
- **Overhead Reduction:** Profile and reduce the ~15-23ms fixed per-frame overhead identified in Assignment 5, now that it's the dominant bottleneck for the fastest model configuration.
- **Advanced Feature Implementation:** Implement one or more of the remaining bonus features not yet built (heatmaps, speed estimation, congestion/intrusion detection, alerting).
- **Pipeline Stress-Testing:** Stress-test the full pipeline against a longer, more realistic video (crowds, occlusion, camera motion) rather than short test clips, to see which of this week's fixes hold up.